MIT 8103 ADVANCED DATABASE SYSTEMS

PORTFOLIO SUMMARY

Name: Lydia Ojoawo

Student ID: 301926017

Case study: FundWise (simulated investment and wallet platform)

Github Repo: github.com/Lydia02/advanced-database-portfolio

CASE STUDY

FundWise is the name of the organization. It is a simulated investment and wallet platform (Bank /Fintech category). It uses no real data. The users include Investors, operations staff, compliance and audit, admin. **The major data are;** users and KYC status, wallets, a transaction ledger, transfers, investment products with daily prices, orders, holdings, an activity feed. **Key operations:** register, deposit, withdraw, transfer, reverse a failed

transfer, buy and sell, audit a balance against the ledger, retrieve a portfolio. **Stack:** PostgreSQL 16 for money, MongoDB 7 for read-heavy views, Docker.

PORTFOLIO 1

Database Design and Modelling

I designed 8 tables in third normal form: users, wallets, transfers, transactions (the ledger), products, product_prices, orders and holdings. Each has a BIGINT identity key. Composite unique keys cover (user_id, product_id) and (product_id, price_date). CHECK constraints block negative balances and transfers to the same wallet. transactions.transfer_id is nullable because deposits have no transfer. I kept wallets.balance and transactions.balance_after as stored copies on purpose. A transaction keeps them correct (Portfolio 3). This makes balance reads cheap and puts the risk in writes.

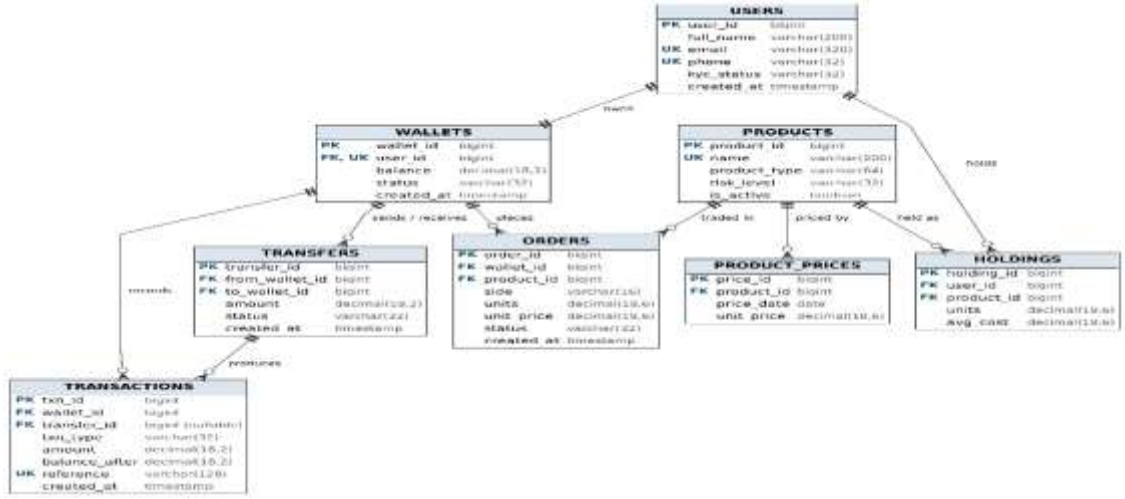

Figure 1. FundWise ER diagram. All columns are NOT NULL except transactions.transfer_id

PORTFOLIO 2

Query Processing and Optimisation

I loaded 10,000 users and wallets, 226,110 ledger rows and 50,000 orders. I ran EXPLAIN ANALYZE five times per query before and after indexing and kept the fastest run. Three indexes turned scans into index lookups and cut the three filtered queries by 141 to 4,840 times. Q5 got no index because it totals about half the ledger, and an index would only slow writes. Indexes help selective queries, not full aggregations.

Query	Before	After	Faster
Q1 wallet statement	Parallel Seq Scan, 150 ms	Index Scan, 0.031 ms	4,840x
Q2 transfers by wallet	Seq Scan, 5.1 ms	Bitmap Heap Scan, 0.036 ms	141x
Q3 orders by wallet and date	Seq Scan, 37 ms	Bitmap Heap Scan, 0.038 ms	980x
Q5 top depositors	Seq Scan, 195 ms	Seq Scan, 123 ms	1.6x

Table 1. Query plans and times before and after indexing (seed data; later demos added a few rows).

PORTFOLIO 3

Transactions and Concurrency

A transfer_funds function writes the transfer, both wallet updates and both ledger rows together. A 1,000 transfer committed on both wallets. A forced rollback restored wallet 4500. An overdraft failed on the wallets_balance_check constraint and left no rows behind. Stored balances still matched the ledger afterwards.

Two sessions then read a balance of 1,000 and withdrew 200 and 300. The result was 700, not 500, with no error. SELECT ... FOR UPDATE made the second session wait, read 800 and end at 500. Under READ COMMITTED a transaction saw 1,000 then 700 for one row. Under REPEATABLE READ it kept 1,000, and its later write failed with SQLSTATE 40001, so the application must retry. Money rows need explicit locking (PostgreSQL Global Development Group, 2024).

PORTFOLIO 4

NoSQL and Advanced Data Models

MongoDB holds a portfolios collection (10,000 documents, holdings embedded) and an activity_feed collection (100,000 documents, payload varies by event). A portfolio page is one lookup by _id instead of a four-table join. I ran lookups, a latest-unread query, two aggregations and an update. With its index the feed query examined 8 documents in 6 ms. Without it, it examined 100,000 documents in 116 ms. MongoDB does not suit moving money, so PostgreSQL stays the source of truth. The Mongo data is generated, not exported, so its balances differ from PostgreSQL (MongoDB, Inc., 2024).

PORTFOLIO 5

Distributed and Cloud Database Exercise

Built and tested locally in Docker (Docker, Inc., 2024). PostgreSQL ran one primary and one streaming replica (asynchronous). A row written on the primary appeared on the replica. I stopped the primary and promoted the replica with pg_promote(). It then accepted writes. Promotion was manual, and a

commit not yet shipped would be lost in a real crash. A three-node MongoDB replica set elected a new primary on its own when I stopped the primary.

Proposed, not built: sharding the ledger on a hashed wallet_id, where cross-shard transfers need distributed transactions, and a managed cloud PostgreSQL with a standby in a second zone. I did not deploy to a cloud provider.

EVIDENCE

Scripts, notes and screenshots are in the repository: Portfolio 1, Portfolio 2, Portfolio 3, Portfolio 4 and Portfolio 5.

INDIVIDUAL REFLECTION

To make reading data faster, I stored the account balance directly on the wallet as well as in the transaction ledger. This taught me that a design choice can move risk from reading data to writing it. I had to be extra careful to keep the two numbers from drifting apart. Concurrency was the hardest part. In my test, two overlapping sessions withdrew 200 and 300 from a starting balance of 1,000. Without proper controls, both succeeded with no error, leaving a wrong balance of 700. I fixed this with SELECT ... FOR UPDATE, which locks the row during the update. The final balance was then correct at 500. I also hit setup problems: a data type error (a numeric cast) in my seed script, and Docker running out of memory. I fixed these one by one.

The biggest surprise was Query 5. Adding an index barely changed its speed. I learned that indexes only help when a query filters down to a small set of rows. With more time, I would deploy this setup to a managed cloud service and see how it performs there.

REFERENCES

MongoDB, Inc. (2024). MongoDB manual. https://www.mongodb.com/docs/manual/

PostgreSQL Global Development Group. (2024).

PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/

Docker, Inc. (2024). Docker documentation. https://docs.docker.com/

Lucid Software Inc. (n.d.). Lucidchart. https://www.lucidchart.com/

pgAdmin Development Team. (n.d.). pgAdmin 4 documentation. https://www.pgadmin.org/docs/

AI-USE DECLARATION

I used Claude (Anthropic) to help develop ideas and through initial drafts, while Lucid AI assisted with the initial ER diagram generation. I redrew the final diagram using Graphviz. I tested the scripts in pgAdmin, mongosh, and Docker, reviewed the outputs, corrected errors, and verified the final work. I understand the work and take responsibility for its accuracy.